Session 6B: SQLite and CRUD

Trainer Reference Guide

Purpose and outcomes

Persist records in SQLite, create tables, use parameterized statements, commit/rollback safely, and isolate data access in a repository.

Examples and exercises

- 01_sqlite_intro.ipynb uses an in-memory database for independent, repeatable CREATE/INSERT/SELECT practice.
- database.py provides a project-safe connection context manager, row mapping, commit, rollback, close, and initialization.
- 02_crud_sqlite.py implements create, list, update, and delete functions outside any route.
- exercise_01_create_read.py is guided initialization/insertion/listing. exercise_02_update_delete.py handles missing IDs. challenge_repository.py moves CRUD into a class without UI calls.

Trainer key points specific to this module

Draw table, row, column, primary key, and unique constraint. Always demonstrate placeholders rather than string-formatted SQL. Explain transaction boundaries and why ‘commit‘ matters. Delete only a disposable lesson database when demonstrating reset.

Portal connection and evidence

The final SQLAlchemy repository preserves these responsibilities. Learners demonstrate CRUD and verify changes survive reconnecting.

Delivery pattern

1. Explain the idea briefly and ask learners to predict a result.
2. Run the simple example, then connect it to an employee/user workflow.
3. Pair learners for the guided task; require an individual attempt next.
4. Use the challenge as the cumulative project increment.
5. Review the quiz and ask learners to explain, not only show, their code.

Trainer key points

- Keep live coding visible and deliberate. Normalize reading tracebacks.
- Ask what type and value each expression produces before running it.
- Let TODO files remain student-owned; open solutions only during review.
- Check boundary and failure cases, not only the happy path.
- End by tracing how the topic appears in the final portal.